

DigiRL：用自主强化学习训练 “真实环境”里的设备控制智能体

精读导读（中文讲解版）

原论文： *DigiRL: Training In-The-Wild Device-Control Agents with Autonomous Reinforcement Learning*

Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, Aviral Kumar

UC Berkeley UIUC Google DeepMind

NeurIPS 2024 | 引用数 177 (Semantic Scholar, 2026-06-07) | 代码：

<https://github.com/DigiRL-agent/digirl>

这篇在解决什么（一句话定位）：让一个不算大的视觉语言模型（VLM）学会“真刀真枪”地操作安卓手机界面（点击、滑动、打字），办法是让它自己在真实安卓环境里反复尝试、靠成功/失败的反馈用强化学习（RL）变强，而不是只靠人类示范去模仿。结果一个 1.3B 的小模型把任务成功率从 17.7% 提到 67.2%，反超了当时用 GPT-4V、Gemini 1.5 Pro 包装的大模型智能体，而且全程不需要额外人工标注。

读前小白知识卡（先把这些概念搞懂）

- **智能体 (agent)**：能感知环境、自己决定下一步动作、并对结果负责的程序。这里的“环境”就是一部安卓手机，“动作”就是点哪儿、滑哪儿、打什么字。
- **VLM (视觉语言模型, vision-language model)**：既能看图、又能读懂/生成文字的大模型（如 GPT-4V、Gemini）。本文的智能体大脑就是一个 VLM：输入是手机截图 + 任务指令，输出是一个动作。
- **GUI (图形用户界面, graphical user interface)**：就是我们平时用的带图标、按钮、输入框的界面。“设备控制”指直接在 GUI 上按像素坐标点点划划，而不是调 API。
- **In-the-wild (真实/野外环境)**：不是实验室里固定不变的模拟器，而是真实的 App 和网站——它们会改版、会弹广告、会让你登录、搜索结果顺序还会变。这种**随机性 (stochasticity)**和**非平稳性 (non-stationarity, 环境随时间在变)**正是本文要对付的难点。
- **模仿学习 / SFT (监督微调, supervised fine-tuning)**：拿人类操作的“标准答案轨迹”让模型照着学。问题：人类数据是**过时的 (stale)**，而且模型一旦走到示范里没出现过的“没见过的状态”

就不知道怎么办、爬不出来。

- **强化学习 (RL, reinforcement learning)**: 让模型自己去试, 做对了给奖励 (reward), 做错了不给, 靠这个信号逐步学会更好的策略 (policy)。本文的奖励很简单: 整件任务做成了给 1, 没做成给 0。
- **在线 / 离线 RL (online / offline RL)**: 在线 = 模型一边实际操作环境一边学 (数据是它自己当下产生的); 离线 = 只拿一批事先收集好的旧数据来学, 不再实际交互。本文是先离线、再“离线转在线” (offline-to-online) 两段式。
- **轨迹 (trajectory / rollout)**: 智能体从开始到结束完成 (或没完成) 一个任务, 中间走过的“状态—动作”序列。一条轨迹就是一次完整尝试。
- **信用分配 (credit assignment)**: 一条轨迹有很多步, 最后只有一个总的成败信号。到底是哪一步走对了/走错了导致了成功/失败? 把功劳或锅分到具体某一步上, 就是信用分配问题。
- **优势函数 (advantage)**: 衡量“在某个状态下做了某个动作, 比平均水平好多少”。优势为正 = 这个动作值得鼓励。本文核心就是挑出优势大的动作让模型去学 (advantage-weighted)。
- **价值函数 (value function)**: 估计“从某状态出发、按当前策略走下去, 最终能拿到多少回报”。本文用了两个: 指令级 (判断整条任务值不值得学) 和步级 (判断某一步好不好)。
- **AWR (优势加权回归, advantage-weighted regression)**: 一种简单又稳的离线 RL 算法。直觉: 把策略往“优势大的动作”那边拉, 同时别离原来的策略太远。本文以它为骨架。

1 摘要全译

视觉语言模型 (VLM) 的训练语料通常缺少足够的、以“决策”为中心的数据。这导致现成的 VLM 在决策类任务上表现欠佳——比如通过图形用户界面 (GUI) 在真实环境中控制设备。虽然“用静态示范数据训练”已显示出一定潜力, 但我们指出: 这类方法不足以控制真实的 GUI, 因为它们无法应对真实世界中的随机性与非平稳性——而这些是静态观测数据里没有体现的。本文提出一种新颖的自主强化学习方法, 称为 **DigiRL**, 用于训练真实环境下的设备控制智能体; 它通过两个阶段微调一个预训练 VLM: 先用离线 RL 来初始化模型, 再做“离线转在线”RL。为此, 我们搭建了一个可扩展、可并行的安卓学习环境, 并为其配备了一个基于 VLM 的评估器 (evaluator); 同时为这一领域设计了一套简单却有效的 RL 方法。我们的方法运行**优势加权 RL**, 其中的优势估计器经过增强以应对随机性, 并配有一个**自动课程 (automatic curriculum)**以榨取最大的学习信号。我们在 Android-in-the-Wild (AitW) 数据集上验证了 DigiRL: 用 RL 训练的 1.3B VLM 取得了 **49.5% 的绝对提升**——成功率从 17.7% 升到 67.2%——超过了用静态人类示范数据做监督微调的版本。这一结果不仅显著超越此前最好的智能体 (包括用 GPT-4V 的 AppAgent, 成功率 8.3%; 以及用 AitW 数据训练的 17B CogAgent, 38.5%), 也超过了此前最好的、基于“过滤式行为克隆”的自主 RL 方法 (57.8%), 从而为真实环境下的设备控制数字智能体确立了新的最先进水平 (SOTA)。

2 逐节精读

2.1 引言：为什么“包装大模型”和“纯模仿”都不够（第 1 节）

愿景。作者想要的是一个真正自主的数字助手：能听懂自然语言、自己在电脑/手机上替你订机票、调 PPT、操作各种 App——而且要能在充满随机性和干扰的真实环境里稳定干活。

核心痛点：会“推理”不等于会“干活”。哪怕是最强的闭源 VLM (GPT-4V、Gemini 1.5 Pro)，在设备上执行任务时也经常给出错误动作。它们在抽象推理上很强，但这种能力迁移不到“低层设备控制”上。所以以往大多数做设备智能体的工作，都是给闭源 VLM 套上一层复杂的“外壳 (wrapper)”：加提示词、加搜索、加工具调用。问题在于：**只要不更新模型权重，外壳能力再花哨，最终天花板也被底座模型卡死。**文中给的反例：让 GPT-4V “去 newegg.com 搜 alienware area 51 并选第一个”，它却跑去 Google 搜了 “alien area 51 ebay”，还自以为完成了任务（见图 2、图 15）。

另一条路——纯模仿 (imitation/SFT) 也不行。网页和设备是动态的，照着过时的示范数据学，环境一变就失效；而且这种智能体无法从自己的错误里爬出来——一旦走进示范里没见过的状态就卡死。

本文主张：让模型在真实设备上从“自己的经验”里学。但这条交互式学习路线有两个硬性要求 (desiderata)：

1. 必须用**在线交互数据**，因为静态示范代表不了部署时的真实情况（网站经常改版、有干扰）。
2. 必须能从**模型自己产生的多轮交互数据**里学，而这里面很大一部分是失败的。所以要有机制**自动挑出对的动作、滤掉错的**。

贡献。提出自主 RL 方法 DigiRL（即“给数字智能体用的 RL”）。两段式：先离线 RL 初始化，再离线转在线 RL 用真实交互数据继续微调。为支撑在线 RL，作者搭了一个**可并行的安卓环境**（最多同时跑 64 个真实安卓模拟器），配一个**基于 VLM 的通用评估器**（对照人类判断的平均错误率仅 2.8%）来自动给奖励。最终：1.3B 模型在 AitW 上比此前 SOTA (18B CogAgent, 38.5%) 高出约 28.7 个百分点（到 67.2%），比此前最好的自主学习方法 (Filtered BC) 高 9% 以上，并大幅超过套在 GPT-4V/Gemini 上的外壳方案 (17.7%)。作者称这是**第一个成功把“离线转在线自主 RL”做到设备控制 SOTA 的工作**。

2.2 相关工作（第 2 节）

- **多模态数字智能体。**与只处理文本/代码的语言智能体不同，控制设备是**像素级、坐标动作空间**的，而且设备 + 互联网生态随机不可预测。以往要么给闭源 VLM 套规则外壳（不微调 = 天花板低），要么用示范数据做模仿学习（过时数据 + 无法纠错）。最接近本文的是“在自采数据上做过滤式模仿学习”的工作。本文是**第一个把自主、离线转在线 RL 扩展到设备控制**的。即便对比那些在简化网页环境 (MiniWob++) 跑在线 RL 的工作，本文**样本效率高约 1000 倍**（约 10^3 条轨迹 vs. 约 10^6 条），且面向真实 GUI。

- **设备控制的环境。**已有的模拟环境大多只为“评测”设计，任务范围窄、完全确定且静态，学不到设备控制需要的丰富技能；另一些虽然任务多但过度简化、迁不到真实世界。本文环境用**自主评估 + Gemini 1.5 Pro**，支持在**并行的真实安卓设备**上做多样、开放式任务，规模远超以往（以往多是单线程模拟器，撑不起大规模在线 RL）。
- **给 LLM/VLM 用的 RL。**多数已有工作只针对**单轮任务**（如偏好优化、推理）。而从专家示范优化单轮，对多步问题往往次优，尤其在高随机/非平稳下。已有的基于价值的多轮 RL 算法通常要维护一堆网络（Q 网络、价值网络、策略网络及其延迟目标），收敛慢、对超参敏感。本文则聚焦**找出能落地的少数关键设计**，做一个简单有效、可被实践者直接用的 RL 算法。

2.3 问题设定与预备知识（第 3 节）

研究范围。基于像素与虚拟设备交互，聚焦**控制安卓设备**——这比以往只做网页导航的环境更难也更通用（浏览器只是设备里的一个 App）。

一个回合 (episode) 怎么走：模拟器从主屏开始 → 从预设的语言指令集里选一个任务 → 智能体操作模拟器去完成。每一步，智能体看到的**观测就是当前屏幕截图**；动作空间沿用前人工作，包括：按归一化 (x, y) 坐标 (0~1 相对屏幕尺寸) **点击/滑动、输入任意长度文本**、以及按 HOME/BACK/ENTER 等特殊键（见图 3）。训练与测试指令来自 AitW 的 General（找信息、用基础 App，如 “In-n-Out 的菜单上有什么？”）和 Web Shopping（网购，如 “去 newegg.com 搜 razer kraken 并选第一个”）两个子集。

随机性带来的挑战。真实设备控制有模拟环境里没有的随机性：(1) 网站/应用频繁更新（非平稳，导致在线观测和旧的离线数据对不上）；(2) 各种不可预测的干扰（弹窗广告、登录请求、搜索结果顺序随机）；(3) 技术故障（网页没加载完、临时访问受限）。

“冻结策略会退化”的实证（图 4）：作者先用 6 月 1~3 日的数据训出一个好模型，然后在 6 月 7~11 日分别测“冻结不更新的策略”和“持续在线更新的策略”。结果**冻结策略的成功率随时间逐渐下滑**（因为网站变了），而**持续在线更新能顶住这种退化**——这正是“必须在线学”的直接证据。

可靠、可扩展的在线 RL 设置。自主 RL 是“边收数据边训练”，要在随机环境里学好，就得有**实时数据收集流水线**收集足够经验来做梯度更新。单线程安卓模拟器因延迟做不到，所以作者**并行化**模拟器（带错误处理）。此外环境必须能给**奖励信号**：判断当前屏幕是否表示任务已完成。作者扩展了 Pan et al. ([26]) 的**端到端自主评估器**——它不需要访问模拟器内部状态、也不需要给每个任务手写规则（这点和那些“给每个任务人工写校验函数”的工作形成对比）。评估器骨干用 **Gemini 1.5 Pro**，用少量示例轨迹+人工标注的成败标签来引导它判断新任务。一个评估器即可覆盖所有 AitW 任务，且与人类标注高度一致（**平均错误率 2.8%**，见图 8）。

2.4 DigiRL 方法：为设备控制量身改造的自主 RL（第 4 节）

把设备控制建成一个有限步长的马尔可夫决策过程 (MDP, Markov decision process): $M = \{S, A, T, \mu_0, R, H\}$ ，用策略梯度求解。开局从初始分布 μ_0 采样初始状态 s_0 和自然语言指令 c ；**只有**

在最后——如果评估器判定任务成功——才给奖励 1，否则给 0；当任务完成或步数超过上限 H 时回合结束。状态用最近两张截图表示。

几个 RL 标准定义（逐符号）：

- **Q 函数** $Q^\pi(s_h, a_h, c) = \mathbb{E}_\pi \left[\sum_{t=h}^H r(s_t, a_t, c) \right]$ ：在第 h 步状态下做动作 a_h 、之后一直按策略 π 走，预期能拿到的长期总回报。
- **价值函数** $V^\pi(s_h, c)$ ：把 $Q^\pi(s_h, a_h, c)$ 对“从 π 采样的动作 a_h ”求平均，即这个状态本身的平均好坏。
- **优势** $A^\pi(s_h, a_h, c) = Q^\pi(s_h, a_h, c) - V^\pi(s_h, c)$ ：某个具体动作比平均水平好多少。

4.1 骨架：用 AWR 做离策略 RL

出发点是 **AWR（优势加权回归）**：把策略往“奖励诱导的指数化优势”那边回归，作为“优化策略梯度但又别离上一版策略太远”的近似：

$$\arg \max_{\pi} \mathbb{E}_{\nu} \left[\log \pi(a | s, c) \cdot \exp(A(s, a, c)/\beta) \right]. \quad (4.1)$$

其中 $\beta > 0$ 是温度超参， ν 是过去经验的分布， $A(s, a, c)$ 是动作的优势。**直觉**：优势越大的样本，权重 $\exp(A/\beta)$ 越大，模型就越被往那个动作上拉。

为了省掉调 β 的麻烦，作者改成对优势做“硬过滤（hard filtering）”——不是按 $\exp(A)$ 连续加权，而是直接挑出优势达标的样本，然后对它们做普通的最大似然（模仿）训练：

$$L(\pi) = -\mathbb{E}_{\text{filter}(\nu)} \left[\log \pi(a | s, c) \right]. \quad (4.2)$$

说人话：先用“优势”这把筛子把好动作挑出来，再让模型像背标准答案一样去学这些好动作。问题是：优势通常靠在环境里跑蒙特卡洛（MC）采样来估，而设备生态太随机，MC 估出来的优势方差很大（同样的动作有时成有时败）。

4.2 用“双稳健”估计器得到可靠的优势（doubly-robust）

为了在强随机下也能可靠分辨“好动作”，作者构造了一个**逐步优势估计器**（受双稳健估计启发）：

$$A^{\text{step}}(s_h, a_h, c) := \lambda^{H-h} r(s_H, a_H, c) + (1 - \lambda^{H-h} r(s_H, a_H, c)) (V^{\text{step}}(s_{h+1}, c) + r(s_h, a_h, c) - V^{\text{step}}(s_h, c)). \quad (4.3)$$

逐符号拆解：

- λ 是一个加权超参； λ^{H-h} 表示离结尾越近、越信任“最终真实结果”这一项。
- 第一项 $\lambda^{H-h} r(s_H, a_H, c)$ 是高方差的蒙特卡洛项（直接看这条轨迹最后成没成，受随机性影响大）。

- 后半括号里的 $V^{\text{step}}(s_{h+1}, c) + r(s_h, a_h, c) - V^{\text{step}}(s_h, c)$ 是**高偏差的自举项**（用学到的步级价值函数估，价值函数拟合不完美会有偏）。
- 因为本问题**中间没有奖励**，这其实是 GAE（广义优势估计）的简化版，只用“下一步优势”和“最终步优势”两端来插值。

核心直觉：把“高方差但无偏”和“高偏差但低方差”两种估计加权融合，找到**方差与偏差的甜蜜点**。实现“步级硬过滤”时很简单：当 $A^{\text{step}}(s_h, a_h, c) > 1/H$ 时，就认为这一步**在朝目标前进**，保留下来。

4.3 用“指令级价值函数”做自动课程 (automatic curriculum)

只有 4.1 的 AWR 更新 + 4.2 的稳健优势，在普通 RL 任务上也许够了，但作者发现对设备控制还不够。原因：任务难度差异极大，在“**模型已经很熟练的任务**”上反复收数据，会拖累样本效率。最有效的学习信号来自**让智能体去经历那些最有信息量的任务**。

于是设计**指令级价值函数** $V^{\text{instruct}}(c)$ ，判断某条轨迹值不值得学：

$$A^{\text{instruct}}(s_h, a_h, c) := \sum_{t=h}^H r(s_t, a_t, c) - V^{\text{instruct}}(c) = r(s_H, a_H, c) - V^{\text{instruct}}(c). \quad (4.4)$$

（等号成立是因为**只有结尾才有奖励**，所以那个求和就等于最终奖励 $r(s_H, a_H, c)$ 。）**直觉：**如果一条轨迹的 A^{instruct} 很大，说明 $V^{\text{instruct}}(c)$ 很小——即这是个**本来很难、但智能体居然做成了**的宝贵经验，应该优先学（类似“优先经验回放/关卡回放”的思想）。训练时先筛出 A^{instruct} **最高的前 p 比例数据**，再丢给 AWR（配 4.3 的稳健优势）。

实现细节：受“现代深度网络（如 Transformer）用**交叉熵损失**训价值函数比用均方误差更好”的发现启发，两个价值函数都用**基于轨迹奖励 MC 估计的交叉熵目标**来训：

$$L(V^{\text{traj}}) = -\mathbb{E}_{\nu} [r(s_H, a_H, c) \log V^{\text{traj}}(c) + (1 - r(s_H, a_H, c)) \log(1 - V^{\text{traj}}(c))]. \quad (4.5)$$

$$L(V^{\text{step}}) = -\mathbb{E}_{\nu} [r(s_H, a_H, c) \log V^{\text{step}}(s_h, a_h, c) + (1 - r(s_H, a_H, c)) \log(1 - V^{\text{step}}(s_h, a_h, c))]. \quad (4.6)$$

说人话：既然奖励只有 0/1，就把价值预测当成“这条任务/这一步最终会成功的概率”，用**二分类的交叉熵**来学。

最终算法 (图 5)：先用收集到的轨迹按 (4.5)/(4.6) 训**指令级**和**步级**两个价值函数；训演员 (actor，即策略) 时，先用 (4.4) 在指令级筛出“值得学的任务”、再用 (4.3) 在步级筛出“走对的那几步”，最后只在**筛过的数据**上用 (4.2) 的 MLE（最大似然）损失更新策略。

2.5 方法图解：DigiRL 总览

图 1 是理解整套方法的关键。它把 DigiRL 拆成三块，请配合图看下面这段“数据怎么流动”的讲解。

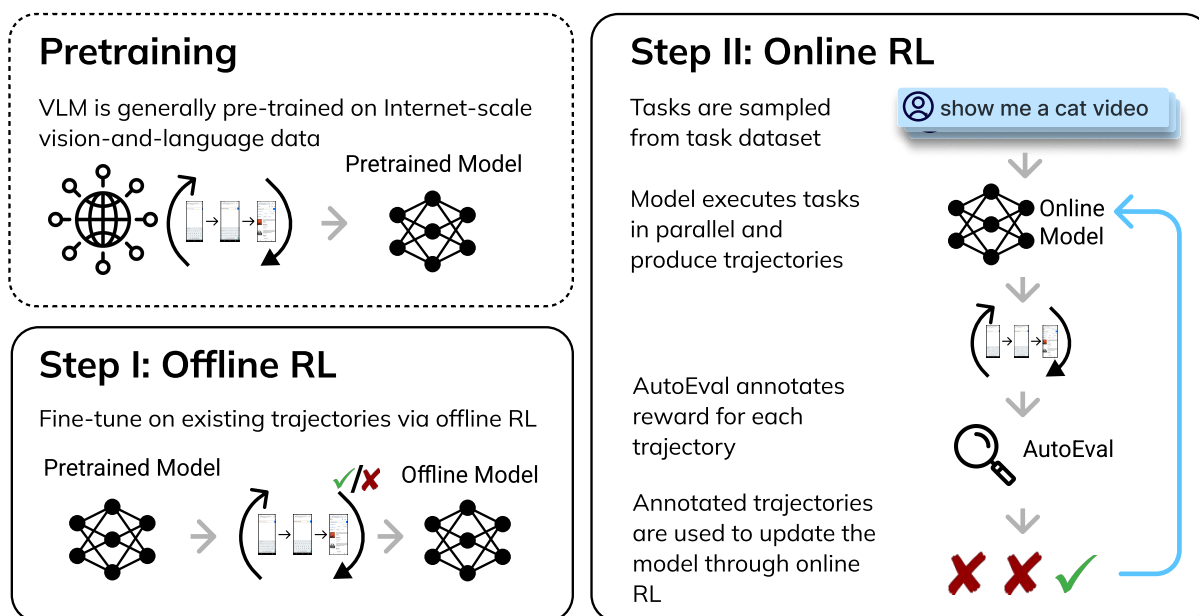


图 1: DigiRL 总览 (原文图 1)。DigiRL 建立在一个已用海量网络数据预训练过的 VLM 之上 (具备常识、推理、视觉定位等基础能力)。首先用离线 RL、拿“过时的任务专属数据”微调 VLM, 先把它的“目标导向行为”引出来; 随后让智能体与真实 GUI 交互, 通过在线 RL 与自主性能评估持续提升表现。

图里在说什么 (顺着箭头走):

- **Pretraining (预训练, 左上虚线框):** 起点是一个在互联网级图文数据上预训练好的 VLM, 得到“Pretrained Model”。它有常识和看图能力, 但还不会“干活”。
- **Step I: Offline RL (离线 RL, 左下框):** 拿一批已有的、可能过时的轨迹, 用离线 RL 微调, 把“Pretrained Model”变成“Offline Model”。图里的对勾/叉号表示用成败信号筛选——这一步先让模型有个像样的起点, 不用真实交互, 省时间。
- **Step II: Online RL (在线 RL, 右侧大框, 整套闭环):** 这是核心循环——
 1. 从任务集采样任务 (图中例子 “show me a cat video”);
 2. **Online Model 并行执行任务**, 产生一批轨迹 (多个安卓模拟器同时跑);
 3. **AutoEval (自动评估器)** 给每条轨迹判成败、打奖励 (红叉 = 失败、绿勾 = 成功);
 4. 把这些自动标注好的轨迹回灌, 用在线 RL 更新模型 (蓝色回环箭头), 然后再去执行——如此往复, 越用越强。

一句话抓住精髓: 整条流水线没有人工标注介入——奖励由 AutoEval 自动给, 数据由模型自己产生, 模型靠这个闭环不断从“自己的成功与失败”里学习, 从而能跟上真实环境的变化。

3 实验解读 (第 5 节)

用什么数据/基准: AitW (Android-in-the-Wild) 的两个子集——**General** (找信息/基础 App, 最多 10 步) 和 **Web Shopping** (网购, 最多 20 步)。General 用全部 545 个训练任务, 测试用测试集前 96 个; Web Shopping 经过后处理 (剔除需要验证码/登录的网站与操作), 训练 438 个、测试 96 个。

对比哪些基线:

- **提示/检索类 (套在闭源 VLM 上):** Set-of-Marks (给可点元素叠数字编号, 让 VLM 直接输出编号而非算坐标) + GPT-4V / Gemini 1.5 Pro; 以及 AppAgent (先让 VLM 探索、把经验加进测试提示)。
- **监督训练类:** CogAgent (18B, 用 AitW 训)、AutoUI (约 1.5B: 200M 语言模型 + 1.1B 视觉编码器)。
- **自主学习类:** Filtered BC (过滤式行为克隆, 此前最强的自采数据方法)。

DigiRL 自己的底座: 基于 AutoUI-Base, **冻结图像编码器**; 价值函数用同一个冻结编码器的视觉特征, 拼上 RoBERTa 抽的指令特征, 再过一个两层 MLP 预测。为公平起见, 纯离线训练用的数据量 $\approx$ 离线转在线训练的总数据量 (两子集离线约 1500 条轨迹; 离线转在线 = 约 500 条离线起步 + 约 1000 条在线)。

主结果（表 1）：小模型大反超

表 1：不同智能体在各设置下的成功率（%）对比（原文表 1，节选）。离线实验跑 3 次报均值 ± 标准差，在线实验跑 2 次；用自主评估器在训练/测试集前 96 条指令上评测。

类别	方法	AitW General		AitW Web Shopping	
		训练	测试	训练	测试
提示	Set-of-Marks · GPT-4V	5.2	13.5	3.1	8.3
	Set-of-Marks · Gemini 1.5 Pro	32.3	16.7	6.3	11.5
	AppAgent · GPT-4V	13.5	17.7	12.5	8.3
	AppAgent · Gemini 1.5 Pro	14.6	16.7	5.2	8.3
监督训练	CogAgent (18B)	25.0	25.0	31.3	38.5
	AutoUI (≈1.5B)	12.5	14.6	14.6	17.7
学习 · 离线	Filtered BC	51.7 ± 5.4	50.7 ± 1.8	44.7 ± 1.6	45.8 ± 0.9
	本文（离线）	46.9 ± 5.6	62.8 ± 1.0	39.3 ± 6.0	45.8 ± 6.6
学习 · 离线转在线	Filtered BC	53.5 ± 0.8	61.5 ± 1.1	53.6 ± 4.7	57.8 ± 2.6
	DigiRL（离线转在线）	63.5 ± 0.0	71.9 ± 1.1	68.2 ± 6.8	67.2 ± 1.5

这些数字说明了什么：

- 光靠提示/检索不够。GPT-4V、Gemini 即便加了 Set-of-Marks，成功率也都不到 20%；AppAgent 这类推理时适配带来的提升不超过 5%。说明不微调，提升空间很有限。失败主要来自“幻觉式推理”——跑到一个相关但错误的页面还自以为对了。
- 监督训练有用但触顶。专门小模型 AutoUI（1.5B）能追平甚至超过通用大模型，但两个子集成功率仍低于 20%；把模型堆到 18B（CogAgent）也还低于 40%——这不是靠堆规模/改架构能根治的。这些智能体最典型的毛病是无法纠正自己的错误（比如误点了语音输入按钮，到步数上限都退不出来）。
- 在线交互是关键。即便离线和“离线转在线”用的样本总量一样，离线转在线显著更强（General 子集 75% vs. 62.8%）——证明让模型实际去环境里交互、从未经筛选的次优数据里学非常有效。
- DigiRL 全面超过 Filtered BC。在 General 从 61.5%→71.9%、在 Web Shopping 从 57.8%→67.2%。综合看：1.3B 的 DigiRL 把成功率从 17.7% 提到 67.2%（绝对 +49.5），反超 18B 大模型和所有外壳方案，且全程零人工标注。

消融与分析 (第 5.2 节)

失败模式分析 (图 7)。把失败分三类：(1) **无法从错误中恢复**（走错后爬不出来，如停在错的搜索页）；(2) **中途卡住**（在正确路上分心，比如没点对链接、打完字忘了搜）；(3) **到了错误目标**（跑到错页面却以为完成了）。三类都因 RL 训练而减少，但**下降最稳、最显著的是“无法从错误中恢复”**——因为预训练模型只会“顺着生成看似合理的下一步”，遇到没见过的状态就抓瞎；而 DigiRL 在**自采轨迹**上训练，学会了从自己的错误里恢复。

各组件消融 (图 9 左，在 Web Shopping 上)。四个设计都必要：

- 价值函数用**交叉熵**（而非回归/均方）：约 **+12%** (Ours vs. Ours w/ Regression)。
- 用**步进优势**：效率约 **+12%** (vs. 去掉步进优势)。
- **自动课程**：学习速度约 **+25%** (去掉步进优势的版本 vs. Filtered BC 对照)。
- 完整版优于**原版 AWR**（没有双稳健优势、没有课程）。此外“**硬过滤**”不掉点——和精调温度 τ 后的指数过滤几乎打平，但实现更简单。

评估器的可靠性 (图 8)。自动评估器报的成功率与人类评估**几乎一致**，差异 $< 3\%$ ；Web Shopping 上更准（语言表达更规整、歧义少），General 偏口语化所以略难。

并行模拟器的加速 (图 9 右)。随机器数量**近线性加速**：分布式模拟器在 128 CPU 上可稳定跑 64 个并行模拟器；而“所有并行都挤在同一台服务器”的朴素基线差很多（64 CPU 下 0.74 vs. 1.74 轨迹/分钟）。

附录里的额外发现

- **步长上限 (附录 B.1)。**给更“富余”的步长 ($H=20$ vs. 10) 能让两种方法都学得更快（一条轨迹里试错机会更多）；两种设置下 DigiRL 都比 Filtered BC **快约 100 条轨迹**。
- **轨迹更短 (附录 B.2、表 4)。**DigiRL 的平均轨迹长度一直比 **Filtered BC 短**——说明它解题更高效、更“直奔目标”。

3.1 讨论与局限 (第 6 节)

作者用 DigiRL 在 AitW 多个安卓控制任务上确立了新 SOTA：先搭可扩展、可并行、带稳健 VLM 评估器的安卓环境支撑快速在线收数据；再做“离线 RL 预训练 + 自主 RL 微调”，在真实互联网/设备生态的随机性中靠交互学习。相对此前 SOTA 实现了**约 280% 的相对提升**（成功率从 17.7% 到 68.2%）。**局限**：受算力所限，目前只在 AitW 的任务上训练，而非设备上所有可能任务（尽管并行模拟器和自动评估器都能扩展到更复杂任务）；算法刻意追求**实现简单**，希望成为后续研究（算法层面、以及扩展任务空间）的**基线**。

4 核心 takeaways (给 AI 应用开发者的启示)

1. “**包装大模型**”有天花板，闭环学习才有上限。给 GPT-4V/Gemini 套提示词/检索/工具，短期见效，但不更新权重，能力被底座卡死。要让智能体在垂直场景里真正变强，往往得让它**从自己的交互反馈里学**。
2. **真实环境会变，所以模型也得一直学**。图 4 的“冻结策略退化”是个强警示：网站/App 一改版，离线训练好的智能体就开始失效。**生产级 agent 需要持续在线适配 (continual/online adaptation)**，而不是一次训练终身使用。
3. 用一个 VLM 当“自动评委”是性价比极高的工程套路。不写每任务的规则、不靠人标，用强 VLM (这里是 Gemini 1.5 Pro) + 少量示例就能给出与人类一致 (错误率 2.8%) 的成败判断——**这把“RL 没有奖励信号”这个最大障碍解决了**，且可扩展到很多任务。做 LLM-as-judge / 自动评测时可借鉴。
4. **小模型 + 好方法 > 大模型 + 纯提示**。1.3B 反超 18B 与各种外壳，关键在**两段式训练 (先离线起步、再在线精炼)**和**把成败信号正确分配到具体步骤**。算力有限时，方法和数据闭环比单纯堆参数更划算。
5. **把工程难点也当成研究的一部分**。能做在线 RL 的前提是**并行、实时、带容错的环境** (这里是 64 个并行安卓模拟器，近线性加速)。**数据收集速度往往才是真正的瓶颈**——基础设施做对了，算法才跑得起来。
6. **保持简单、可复现**。作者特意不堆一大堆 Q/V/策略网络，而是用 AWR 这种“筛出好动作再模仿”的简单骨架，加两三个关键改造 (双稳健优势、交叉熵价值、自动课程)。**对工程落地而言，简单稳健、超参不敏感，比花哨更重要**。

参考文献 (节选最关键几条，英文原样)

1. C. Rawles, A. Li, D. Rodriguez, O. Riva, T. Lillicrap. *Android in the Wild: A Large-scale Dataset for Android Device Control*. arXiv:2307.10088, 2023.
2. J. Pan, Y. Zhang, N. Tomlin, Y. Zhou, S. Levine, A. Suhr. *Autonomous Evaluation and Refinement of Digital Agents*. arXiv:2404.06474, 2024.
3. X. B. Peng, A. Kumar, G. Zhang, S. Levine. *Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning*. arXiv:1910.00177, 2019.
4. Z. Zhang, A. Zhang. *You Only Look at Screens: Multimodal Chain-of-Action Agents (AutoUI)*. 2023.
5. W. Hong et al. *CogAgent: A Visual Language Model for GUI Agents*. 2023.

6. J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel. *High-Dimensional Continuous Control Using Generalized Advantage Estimation (GAE)*. 2018.
7. J. Farebrother et al. *Stop Regressing: Training Value Functions via Classification for Scalable Deep RL*. 2024.